ASSIGNMENT : 3

Computer Organisation And Architecture

NAME : GURLEEN
COURSE : B.TECH CS (DS)
ROLL NO. : 2301420051

1. Clocking Methodology :-

→ Clocking Methodology refers to the way a processor uses a clock signal to synchronize all operations. Every component in the CPU performs actions at specific clock edges, ensuring coordinated executed.

- Uses a periodic clock signal.
- Synchronizes data transfer between components.
- Prevents timing conflicts and data corruption.

2. What is a clock cycle? Why is it important?

→ A clock cycle is the smallest unit of time in a processor during which a basic operation can occur. It defines how fast a CPU can execute instructions.

1. Measured in nanoseconds (ns).
2. CPU speed = no. of clock cycles per second (GHz).
3. Each instruction may take 1 or more cycles.

3. Amdahl's Law :

→ Amdahl's Law explains the maximum improvement in performance when part of a system is enhanced (like parallel processing). It shows that overall speedup is limited by the portion that cannot be parallelized.

$$\text{Speedup} = \frac{1}{(1-P) + \frac{P}{S}}$$

→ where,

P : Parallel portion

S : Speedup of parallel part.

4. Data Path :

→ The Data Path is the part of the processor that handles data processing and movement. It includes all hardware elements that perform arithmetic and logical operations.

- Contains ALU, registers, buses, multiplexers.
- Responsible for execution of instructions.
- Works under control unit signals.

5. Control Unit :

→ The control unit acts as the brain of the processor, directly all operations by generating control signals. It controls ALU, memory, I/O operations, interprets instructions from memory and generates time & control signals.

6. Single - Cycle Processor :

→ A single - cycle processor executes one complete instruction in a single clock cycle. All steps like fetch, decode, execute, memory access and write - back happen in one cycle. It is simple and easy to design but not efficient for modern high - performance systems.

7. Multi - cycle Processor :

→ A multi - cycle processor divides instruction execution into multiple smaller steps, each completed in a separate clock cycle. Each stage (fetch, decode, execute, etc takes one cycle. Same hardware reused across cycles.

8. Instruction Pipelining :

→ Instruction pipelining is a technique where multiple instructions are executed simultaneously by overlapping different stages of execution. Instead of completing one instruction before starting another, the processor works like an assembly line.

→ Fetch → Decode → Execute → Memory → WriteBack

It improves throughput and does not reduce individual instruction time, but increases efficiency.

9. Instruction - Level Parallelism :

→ It refers to the processor's ability to execute multiple instructions at the same time within a pipeline. It is achieved using pipelining, superscalar execution, improves CPU performance without increasing clock speed & limited by dependencies between instructions.

10. Data Hazards :

→ A data hazards occurs when an instruction depends on the result of a previous instruction that has not yet completed in the pipeline.

→ Types : Read After Write (RAW)
Write After Read (WAR)
Write After Write (WAW)

- It causes incorrect results if not handled.

11. Control Hazards :

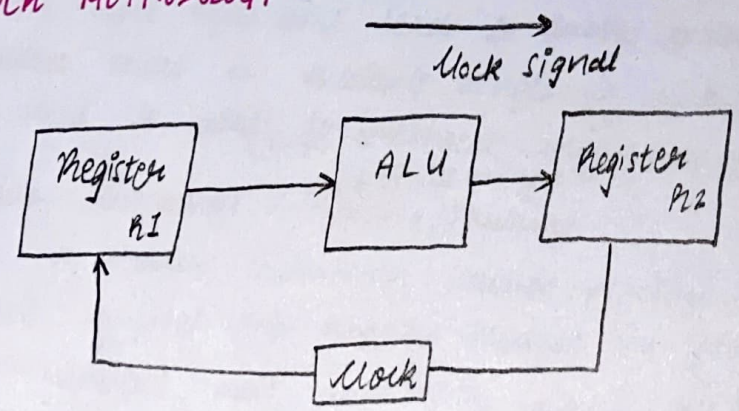
→ Control Hazards occurs due to branch instructions that change the normal flow of execution in a pipeline. It happens in instructions like if-else, loops, jumps and pipeline may fetch wrong instructions. It affects overall performance significantly.

12. Hazard Mitigation :

→ Pipeline hazards can be reduced using various techniques to maintain smooth execution.

- Forwarding : Directly passes result to next instruction.
- Pipeline Stall : Pauses execution until data is ready.
- Branch Prediction : Predicts next instruction path.
- Instruction Reordering : Rearranges instructions to avoid conflicts.

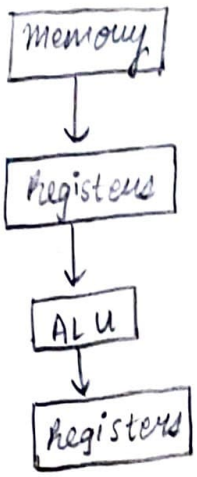
1. CLOCK METHODOLOGY :



→ Clock signal synchronizes all components. Data moves between registers and ALU only at specific edges.

1. It ensures timing consistency.
2. Prevents race conditions.

Data Path :



- Data Path is responsible for data movement and processing. Instructions are fetched from memory, processed in ALU and in results stored back. It works under control unit signals and it is core of instruction execution.

Instruction Pipelining Stages :

	1	2	3	4	5	6		
I_1	IF	ID	EX	MEM	WB			
I_2		IF	ID	EX	MEM	WB		
I_3			IF	ID	EX	MEM	WB	
I_4				IF	ID	EX	MEM	WB

- Multiple instructions execute in overlapping stages, improving throughput and increases efficiency, not speed of single instruction and works on parallelism and can cause hazards.

* Single - cycle Processor :

IF → ID → EX → MEM → WB (one cycle)

Fetch → Decode → Execute → Memory → Write Back

* Multi - cycle Processor :

cycle 1 : IF

cycle 2 : ID

cycle 3 : EX

cycle 4 : MEM

cycle 5 : WB

Multi-cycle splits execution into smaller steps across cycles.

- Frequent Data Dependencies:

When a processor pipeline has frequent data dependencies, the main problem that occurs is a data hazard. This happens because an instruction needs the result of a previous instruction that is still being processed, leading to incorrect execution or delays.

Mitigation Techniques:

- Forwarding (Bypassing): Directly pass result to next Instruction without waiting.
- Pipeline stall: Pause Pipeline until data is ready.

- Amdahl's Law:

Given,

Parallel Portion $P = 0.8$

Speedup of parallel part $S = 4$

$$\text{Speedup} = \frac{1}{(1-0.8) + \frac{0.8}{4}} = \frac{1}{0.2 + 0.2} = \frac{1}{0.4} = 2.5$$

Overall Speedup $= 2.5$

3. Why Multi-cycle is more efficient than single-cycle?

→ A multi-cycle processor is often more efficient because it breaks instruction execution into smaller steps, allowing use of hardware and shorter clock cycles.

- Shorter clock cycle time leads to faster operations.
- Hardware reused in multiple stages is cost efficient.
- No need to wait for longest instruction.

→ Branch - Instruction & Pipeline Flushing:

When a branch instruction causes pipeline flushing, it leads to a control hazard. This happens because the processor does not know the correct next instruction until the branch decision is made.

- Wrong Instructions maybe fetched and needed to discard.
- Causes pipeline delays and performance loss.

SECTION : D

1. Pipeline Diagram :

	1	2	3	4	5		
I_1	IF	ID	EX	MEM	WB		
I_2		IF	ID	EX	MEM	WB	
I_3			IF	ID	EX	MEM	WB

- The Execute (EX) stage is where the actual computation happens. The ALU performs arithmetic or logical operations such as addition, subtraction, comparison or address calculation for memory instructions. Performs calculations and decision-making.

2. DataPath Diagram :

- Design Decision : Registers are placed close to the ALU to the allow fast access and minimal delay, instead of directly accessing slower memory every time. Registers act as temporary high-speed storage. and reduce memory access-time and improve execution speed.

3. Data Hazard Diagram :

- I_1 : $R1 = R2 + R3$
- I_2 : $R4 = R1 + R5$ ← depends on R1

→ Pipeline :

I_1 : IF → ID → EX → MEM → WB

I_2 : IF → ID → (wait) → EX → MEM → WB

here,
Instruction 2 needs the value of R1, but Instruction 1 has not written it yet and this causes a RAW (Read After Write) data hazard.

Mitigation technique (Forwarding):
- Result from ALU is directly forwarded to next instruction.
- No need to wait for write-back stage.
- Avoid pipeline stall.

Forwarding reduces delay, but increases hardware complexity, showing a trade-off between speed and design-cost.